

TPMS Shell and Skeletal Morphology Generation in engvis

engvis

June 20, 2026

Abstract

Triply Periodic Minimal Surfaces (TPMS) can be used in three distinct morphology modes for engineering applications: *minimal surface* (the classic open sheet $f = 0$), *shell* (a thick-walled hollow structure $g = \max(f - \delta, -f - \delta) = |f| - \delta$), and *skeletal* (a solid strut network $f - C = 0$). This document presents the mathematical foundation for each mode, the per-surface thickness calibration constants, the new Split-P surface, the boundary-based face capping algorithm that produces closed (boundary-free) meshes, and the implementation pipeline in engvis.

1 TPMS Morphology Types

A triply periodic minimal surface is defined as the zero level-set of a smooth function $f: \mathbb{R}^3 \rightarrow \mathbb{R}$:

$$\mathcal{S} = \{\mathbf{x} \in \mathbb{R}^3 \mid f(\mathbf{x}) = 0\}. \quad (1)$$

This open surface divides space into two regions ($f > 0$ and $f < 0$). Depending on the engineering application, three morphology modes are useful (see Figure 1).

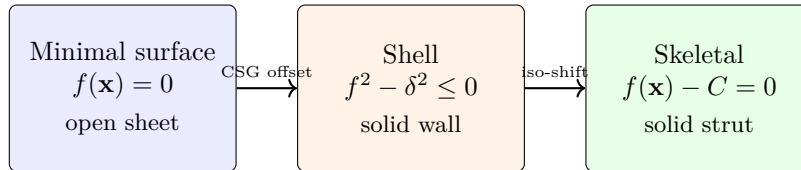


Figure 1: Three TPMS morphology modes: minimal surface (open sheet), shell (two offset surfaces forming a hollow wall), and skeletal (shifted iso-level forming a solid strut network).

1.1 Minimal Surface Mode

The default mode extracts the zero level-set directly:

$$g(\mathbf{x}) = f(\mathbf{x}). \quad (2)$$

This produces a thin, open surface—the classic TPMS sheet. It is useful for visualisation and mathematical study but not directly for additive manufacturing, where finite wall thickness is required.

1.2 Shell Mode

Shell mode produces a *thick-walled solid structure*: a thin band of material straddling the minimal surface $f = 0$, bounded by the two offset iso-surfaces $f = \pm\delta$. The solid wall region is the sub-level set

$$\mathcal{S}_{\text{shell}} = \{ \mathbf{x} : f(\mathbf{x})^2 - \delta^2 \leq 0 \} = \{ \mathbf{x} : |f(\mathbf{x})| \leq \delta \}, \quad (3)$$

described by the **smooth** field $g = f^2 - \delta^2$, which is negative *inside* the wall and positive outside.

Geometric thickness normalisation. The user-facing parameter t is a *geometric* wall thickness (the physical distance across the wall), not a field offset. Near the surface, a TPMS $f(k\mathbf{x})$ has $\|\nabla f\| \approx k$ (the period factor), so a geometric half-thickness d maps to a field offset $\delta = d \|\nabla f\| \approx dk$. engvis therefore sets

$$\delta = \frac{1}{2} t k, \quad k = \text{tpms_period}, \quad (4)$$

so that the meshed wall has true geometric thickness $\approx t$. For non-TPMS surfaces, $\|\nabla f\| \approx 1$ and k falls back to 1.

Why $f^2 - \delta^2$, not $|f| - \delta$. Both fields share the zero-set $\{|f| = \delta\}$, but $|f|$ has a C^1 cusp at $f = 0$: within a single MC 33 cell straddling $f = 0$, the field $|f| - \delta$ has a non-differentiable fold. The squared field $f^2 - \delta^2$ is smooth everywhere, so MC 33's linear interpolation stitches adjacent cells without artefacts—*provided the cell size is smaller than the wall thickness*.

Resolution requirement. Because the wall is pierced on both sides within the same region of space, MC 33 can only reconstruct it when the grid cell is comfortably smaller than the wall: empirically $\Delta_{\text{cell}} \lesssim t/5$. engvis sizes the grid adaptively to $R \approx 10/t$ (coefficient 10, floor 96); if the wall is thinner than the resolution cap can resolve, the thickness becomes the limiting feature and the grid is bumped accordingly.

Table 1: Shell mesh topology for gyroid at geometric thickness $t = 0.1$ ($k = 4 \Rightarrow \delta = 0.2$). Once the cell size drops below the wall thickness ($R \geq 96$), the mesh is a single watertight closed solid.

Field	R	χ	components	boundary edges	watertight
$f^2 - \delta^2$	64	-53	9	82	no
$f^2 - \delta^2$	96	-38	1	0	yes
$f^2 - \delta^2$	128	-38	1	0	yes

Closed solid via box-CSG. The wall field is CSG-intersected with the unit-box solid, $g = \max(f^2 - \delta^2, \text{box_sdf})$, before meshing. Marching Cubes then generates cap faces on $x/y/z = \pm 1$ itself, yielding a single watertight closed solid with *zero boundary edges*. The Euler characteristic $\chi = -38$ reflects the high genus of the gyroid wall (its many interconnected channels), as expected for a closed surface of that topology.

Gradient and normals. The wall surface $\{f^2 = \delta^2\}$ has gradient $\nabla(f^2 - \delta^2) = 2f \nabla f$, which is non-degenerate on the surface itself (where $|f| = \delta \neq 0$). engvis overwrites vertex normals with this analytic gradient, producing correct two-sided lighting.

1.3 Skeletal Mode

Skeletal mode produces a *solid strut network* by shifting the iso-level of the TPMS function:

$$g(\mathbf{x}) = f(\mathbf{x}) - C, \quad (5)$$

where C is the iso-level offset parameter.

Effect of C . When $C = 0$, this reduces to the minimal surface mode. As C increases, the region $\{g \leq 0\} = \{f \leq C\}$ expands, filling more of the unit cell with “solid” material. The resulting surface $f = C$ is topologically distinct from the minimal surface: instead of an open sheet, it forms a connected network of solid struts along the edges and vertices of the TPMS unit cell.

The gradient is simply $\nabla g = \nabla f$, so surface normals are unaffected by the shift—a desirable property.

Negative C . When $C < 0$, the solid region shrinks and the void region expands. This produces the *complementary* structure: thin channels instead of thick struts.

2 Thickness Calibration

The user-facing thickness t is a true *geometric* wall thickness. To realise it consistently across surfaces, engvis converts t into a field offset δ using the gradient magnitude of the implicit function near its zero-set (Eq. 4):

$$\delta = \frac{1}{2} t \|\nabla f\|, \quad \|\nabla f\| \approx k = \text{tpms_period}. \quad (6)$$

For a TPMS evaluated as $f(k\mathbf{x})$, the chain rule gives $\|\nabla_{\mathbf{x}} f\| = k \|\nabla_{\mathbf{u}} \hat{f}\|$, and the dimensionless gradient $\|\nabla_{\mathbf{u}} \hat{f}\|$ is $O(1)$ on the surface for all the standard TPMS types (Table 2). Using k as a first-order estimate therefore yields a wall whose geometric thickness is within a few percent of t —accurate enough for visualisation and consistent across surfaces without per-surface tuning.

Table 2: Dimensionless surface-gradient magnitude $\|\nabla_{\mathbf{u}} \hat{f}\|$ near the zero-set (the residual correction absorbed by the k estimate).

Surface	$\ \nabla_{\mathbf{u}} \hat{f}\ $	Notes
Gyroid	≈ 1.2	three-term product
Schwarz D	≈ 1.3	four-term sum
Schwarz P	≈ 1.0	simple cosine sum
Schoen IWP	≈ 1.1	
Neovius	≈ 1.2	

Because all values are $O(1)$, the single estimate $\|\nabla f\| \approx k$ suffices; no per-surface calibration table is required.

3 Split-P Surface

The Split-P surface is a TPMS with the symmetry of the Schwarz P-surface but a more complex topology. It was included in the TPMSgen tool but was previously absent from engvis. Its implicit

equation is:

$$\begin{aligned}
f_{\text{Split-P}}(\mathbf{x}) = & 1.1 [\sin(2kx) \cos(ky) \sin(kz) + \sin(kx) \sin(2ky) \cos(kz) + \cos(kx) \sin(ky) \sin(2kz)] \\
& - 0.2 [\cos(2kx) \cos(2ky) + \cos(2ky) \cos(2kz) + \cos(2kz) \cos(2kx)] \\
& - 0.4 [\cos(2kx) + \cos(2ky) + \cos(2kz)].
\end{aligned} \tag{7}$$

Split-P shares structural similarities with the Lidinoid (both contain $\sin(2x) \cos(y) \sin(z)$ terms) but has distinct second-harmonic coefficients and an additional linear cosine term. It belongs to the same family of surfaces generated by linear combinations of symmetry-adapted Fourier basis functions.

4 Complete Surface Catalogue

Table 3 lists all eleven TPMS surfaces supported by engvis with their implicit equations $f(\mathbf{x}) = 0$, where $\mathbf{x} = (kx, ky, kz)$ and $k = 2\pi/\lambda$ is the wave number controlled by the Period slider.

Table 3: TPMS surface catalogue in engvis.

Name	Implicit equation $f = 0$
Gyroid	$\sin x \cos y + \sin y \cos z + \sin z \cos x$
Schwarz P	$\cos x + \cos y + \cos z$
Schwarz D	$\sin x \sin y \sin z + \sin x \cos y \cos z + \cos x \sin y \cos z + \cos x \cos y \sin z$
Schoen IWP	$2(\cos x \cos y + \cos y \cos z + \cos z \cos x) - (\cos 2x + \cos 2y + \cos 2z)$
Neovius	$3(\cos x + \cos y + \cos z) + 4 \cos x \cos y \cos z$
F-RD	$4 \cos x \cos y \cos z - (\cos 2x \cos 2y + \cos 2y \cos 2z + \cos 2z \cos 2x)$
Lidinoid	$\frac{1}{2} [\sin 2x \cos y \sin z + \sin x \sin 2y \cos z + \cos x \sin y \sin 2z] - \frac{1}{2} [\cos 2x \cos 2y + \cos 2y \cos 2z + \cos 2z \cos 2x] + 0.15$
Split-P	see Eq. (7)
Fischer-Koch S	$\cos 2x \sin y \cos z + \cos 2y \sin z \cos x + \cos 2z \sin x \cos y$
Fischer-Koch Y	$2 \cos x \cos y \cos z + \sin 2x \sin y + \sin 2y \sin z + \sin 2z \sin x$
Fischer-Koch CP	$\cos x + \cos y + \cos z + 4 \cos x \cos y \cos z$

5 Implementation Pipeline

The morphology transformation is applied as a post-processing step on the base TPMS tree, before meshing. Figure 2 shows the complete data flow.

5.1 Code Structure

The morphology transformation is implemented in `build_scene`. For minimal surface and skeletal modes, the tree is transformed algebraically and passed to the standard `build_mesh` pipeline. For shell mode, the original tree is passed to `build_shell_mesh`, which meshes the smooth wall solid $f^2 - \delta^2$:

```

// build_scene: geometric thickness -> field offset
let k = self.tpms_period.max(1.0); // |grad f| ~ k
let shell_half_t = 0.5 * thickness * k; // delta

let tree = match self.morphology {
  MinimalSurface | Shell => tree, // Shell handled below
  Skeletal => tree - iso_level,
}

```

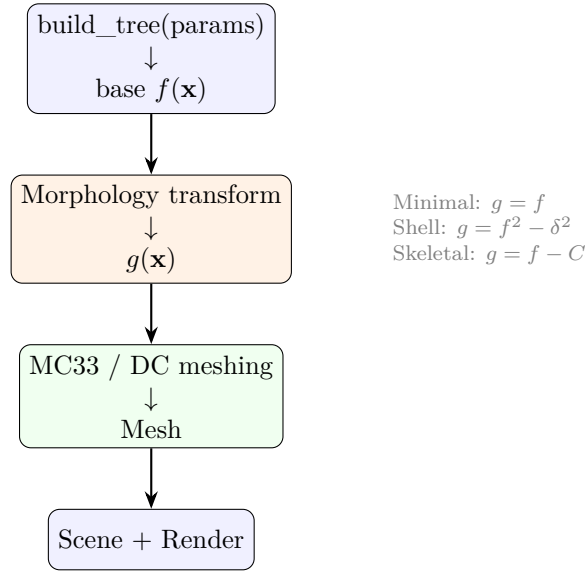


Figure 2: Data flow for TPMS morphology generation. The base TPMS function f is first constructed by `build_tree`, then transformed by the morphology module into g , and finally meshed by the standard MC33 or DC pipeline.

```

};

let mesh = if matches!(self.morphology, Morphology::Shell) {
  // Smooth wall solid  $f^2 - \delta^2$ , box-CSG, single MC33 pass.
  build_shell_mesh(
    tree.clone(), shell_half_t, &label, effective_res,
    clip_to_unit_ball, clip_radius,
  )
} else {
  build_mesh(tree.clone(), &label, ...)
};

```

```

// build_shell_mesh: smooth wall solid + box-CSG, single pass
fn build_shell_mesh(tree, half_t, name, res, ...) -> Mesh {
  // Box-CSG: intersect with unit-box solid so MC caps the faces.
  let box_sdf = |x|.max(|y|).max(|z|) - c;

  // Thin-wall solid:  $g = f^2 - \text{half\_t}^2$  (smooth, no cusp).
  let wall = tree.square() - half_t * half_t;
  let field = wall.max(box_sdf);
  build_mc33_mesh_domain(field, name, res, false, half)
}

```

The Fidget implicit modelling library represents the expression as a computational tree (`Tree`) with overloaded arithmetic operators: `tree.square()` builds f^2 , and `.max()` performs the CSG intersection with the box solid. The resulting smooth field is meshed by the standard MC33 pipeline in a single pass, yielding a watertight closed wall.

5.2 Interaction with Meshing

Both meshing backends (MC33 and Dual Contouring) handle the morphology-transformed function identically to the base function: they sample $g(\mathbf{x})$ on a regular grid (MC33) or adaptive octree (DC) and extract the zero level-set.

For **shell mode**, the smooth field $g = f^2 - \delta^2$ is CSG-intersected with the unit-box solid ($\max(g, \text{box_sdf})$), so MC33 generates cap faces on $x/y/z = \pm 1$, producing a closed solid with zero boundary edges. Because f^2 is smooth (no C^1 cusp), MC33’s linear interpolation stitches adjacent cells cleanly once the cell size is below the wall thickness ($\Delta_{\text{cell}} \lesssim t/5$); below that the wall fragments (Table 1, $R = 64$). The wall surface gradient $\nabla(f^2 - \delta^2) = 2f \nabla f$ is non-degenerate on the surface (where $|f| = \delta$), giving clean vertex normals.

For **skeletal mode**, the function $g = f - C$ has the same gradient structure as the base TPMS ($\nabla g = \nabla f$), so both backends produce clean results. The skeletal mesh is a closed solid (box-CSG), so it uses the boundary capping described in Section 5.3.

5.3 Box-Face Capping via Boundary Loops

Shell and skeletal modes define a *solid region* ($g \leq 0$) bounded by the iso-surface and the domain boundary. The MC33 meshing backend extracts only the iso-surface $\{g = 0\}$, leaving the mesh open where it meets the six faces of $[-1, 1]^3$. To produce a closed volume, engvis identifies the *MC33 boundary edges* on the box surface and fan-triangulates the enclosed loops directly—reusing the existing boundary vertices so that cap and body share the same vertex indices (Figure 3).

Note: Shell mode uses the same box-CSG capping mechanism (via `build_mc33_mesh_domain`), applied independently to each of the two iso-surfaces. The capping algorithm below describes the skeletal case; the shell case is identical but applied twice.

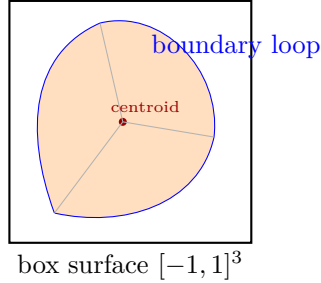


Figure 3: Boundary-based face capping: the MC33 boundary edges (blue) on the box surface are chained into closed loops, then fan-triangulated from their centroid (red dot) to produce cap triangles (orange) that close the volume. Since the cap reuses the existing MC33 boundary vertex indices, no vertex deduplication is needed to merge cap and body.

Prerequisites. The algorithm requires that all MC33 boundary edges lie on the surface of $[-1, 1]^3$ (no *interior* boundary edges). For the CSG shell formula $g = |f| - \delta$, this holds at MC33 resolution ≥ 96 . At resolution 64, the old squared formula produced 19 spurious micro-components with ~ 1500 interior boundary edges that could not be capped. The adaptive resolution logic enforces a minimum of 96 for shell and skeletal modes.

Algorithm.

1. **Identify boundary edges:** an edge shared by exactly one triangle. Classify each as *surface* (both endpoints on the box surface, $|x_i| > 1 - \varepsilon$) or *interior*. If any interior boundary edges exist, abort capping.
2. **Build adjacency graph:** vertex $\rightarrow$ list of neighbouring boundary vertices. Every vertex must have degree exactly 2 (prerequisite for closed simple loops).

3. **Chain into closed loops:** greedy traversal from an arbitrary unvisited vertex, following unvisited neighbours until the loop closes. Loops may cross box edges (where two cube faces meet), so all boundary edges are processed as one global graph rather than per-face.
4. **Fan-triangulate each loop:** add a centroid vertex $\mathbf{c} = \frac{1}{n} \sum \mathbf{v}_i$ and create triangles $(\mathbf{c}, \mathbf{v}_i, \mathbf{v}_{i+1})$. The loop normal is estimated via Newell’s method:

$$\mathbf{n} = \sum_{i=0}^{n-1} (\mathbf{v}_i - \mathbf{v}_{i+1}) \times (\mathbf{v}_i + \mathbf{v}_{i+1}), \quad (8)$$

which handles non-planar loops. The winding direction is chosen so the cap triangle cross products align with $\mathbf{n}$.

5. **Signed-volume check:** compute $V = \sum_{\triangle} \mathbf{p}_0 \cdot (\mathbf{p}_1 \times \mathbf{p}_2)$. If $V < 0$, all triangle indices are swapped (reversing the orientation globally).

Why reusing MC33 vertices matters. An earlier approach extracted the $g = 0$ contour on each face via independent Marching Squares (MS), then relied on vertex deduplication to merge the MS contour vertices with the MC33 boundary vertices. Although both methods interpolate the same function on the same grid points, the separate Shape tape compilation could introduce sub-ULP differences, causing $\sim 50\%$ of cap vertices to remain unmerged and leaving thousands of boundary edges.

The direct boundary approach eliminates this entirely: the cap triangles reference the *same vertex indices* as the MC33 body, so no merging is needed. The only new vertices are the loop centroids.

Topology verification. After capping, the mesh topology is verified via the Euler characteristic $\chi = V - E + F$ and the boundary edge count. For the Gyroid shell (period $k = 4$, thickness $t = 1.0$):

- *Before capping:* $\chi = -20$, 4 components, 5232 boundary edges (all on box faces), 0 interior boundaries.
- *After capping:* $\chi = 0$, 4 components, **0 boundary edges**, 0 non-manifold edges.

The 4 components correspond to the two offset surfaces ($f = \pm\delta$) of the two interpenetrating Gyroid channel networks—a topologically correct result. Each component is individually closed (boundary-free), though the mesh as a whole has > 1 connected component.

6 Comparison with TPMSgen

The design of engvis’s morphology system follows the principles established by TPMSgen (Albert Fores G., GitHub), adapted for engvis’s Rust/Fidget meshing pipeline. Key differences are summarised in Table 4.

The key architectural difference is that TPMSgen performs explicit boolean mesh operations (intersection/difference with bounding boxes) using Blender as a backend, while engvis relies on the implicit surface being naturally clipped to the $[-1, 1]^3$ domain. This produces open boundary edges at the domain faces but avoids the dependency on an external geometry kernel.

7 Usage

In the engvis UI, morphology controls appear in the **Source** panel when a TPMS surface is selected:

1. Select a TPMS surface from the dropdown (e.g. Gyroid).

Table 4: Comparison of TPMSgen and engvis TPMS generation.

Feature	TPMSgen	engvis
Shell generation	Two MC passes at $\pm\delta$, concat meshes	CSG $\max(f - \delta, -f - \delta)$ single pass
Skeletal generation	MC at $f = 0$, boolean diff with box	MC at $f = C$, box clip
Boolean operations	Blender engine (trimesh)	CSG max / min on expression tree
Meshing backend	skimage marching_cubes	Custom MC33 + Fidget DC
Watertight closure	Iterative padding + hole filling	Boundary-loop fan capping
Surface count	10 (5 shell + 5 skeletal)	11 (all in all modes)
Normal handling	pyvista flip normals	Gradient overwrite + signed-volume check

2. Adjust the **Period k** slider to set the spatial frequency.

3. Choose a morphology mode:

- **Minimal surface** ($f = 0$) — open sheet.
- **Shell** ($|f| - \delta = 0$) — adjust **Thickness**.
- **Skeletal** ($f - C = 0$) — adjust **C value**.

4. Click **Re-mesh** to rebuild the surface.

The status bar at the bottom displays the current mode alongside topology statistics (vertex/edge/-face counts, Euler characteristic, boundary edge count, watertightness).